

AWS – Strands Agents

Homework

AWS – Strands Agents.....	1
Homework.....	1
Required to apply	1
Final submission	2
Example	2

Goal: create a small but complete Strands Agents prototype that demonstrates practical use of agentic concepts.

You may choose any domain, for example: travel assistant, study planner, code reviewer, document summarizer, expense advisor, or project artifact analyzer.

Required to apply

The homework should connect to the course topics:

1. **Basic agent anatomy:** model, prompt/system instructions, tools, memory/state.
2. **Tools and community tools:** use at least one tool from strands-agents-tools or another community-provided tool package. Strands community tools provide ready-made capabilities such as file operations, shell/system tools, API access, and math utilities.
3. **MCP integration:** connect at least one MCP server, for example filesystem, GitHub, web/search, or custom local MCP.
4. **Skills:** create and use at least one skill, for example “summarize document”, “extract requirements”, “validate JSON”, “prepare report”.
5. **Steering:** add steering instructions that control agent behavior, style, constraints, or safety boundaries.
6. **Hooks:** add at least one hook, for example logging before/after tool calls, counting tokens, saving intermediate results, or validating output.
7. **Interrupts:** demonstrate one human-in-the-loop interruption, for example approval before writing a file or before sending final output.
8. **Retries:** add retry logic for failed tool calls, malformed output, or low-quality validation results.

9. **Multi-agent pattern:** use one pattern: Workflow, Graph, Swarm, or Supervisor/agents-as-tools. Strands documentation presents Graph, Swarm, and Workflow as the main multi-agent orchestration options.
10. **Evaluations:** create several test cases and evaluate agent output.
11. **Observability:** collect traces, metrics, and logs. The article project also emphasizes observability through CloudWatch metrics and alarms as part of operating Strands-based systems in AWS.

Final submission

Should have:

1. GitLab repository.
2. README.md with setup and run instructions.
3. Architecture diagram.
4. Explanation of used Strands concepts.
5. Screenshots of execution.
6. Screenshots from AWS console.
7. Evaluation results.
8. Short reflection: what worked well, what was difficult, and what they would improve.

Example

Personal Study Planner Agent

Domain: planning a simple study session.

Input example:

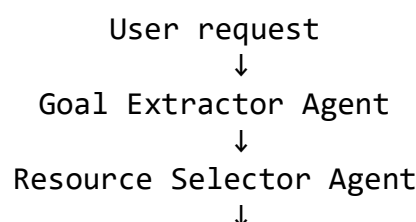
I have 2 hours tomorrow. I need to study Java collections and prepare for a short quiz.

Output:

Study plan:

1. 20 min – review List, Set, Map
2. 30 min – solve practice tasks
3. 20 min – compare ArrayList vs LinkedList
4. 30 min – quiz preparation
5. 20 min – recap and self-check

3-agent version



Plan Writer Agent



Final study plan

Agent 1: Goal Extractor Agent

Purpose: understand what the student wants to learn.

Extracts:

```
{
  "topic": "Java collections",
  "available_time": "2 hours",
  "goal": "prepare for quiz",
  "constraints": ["tomorrow"]
}
```

Uses:

Skill: `extract_learning_goal`

Steering: do not invent topics

Retry: if JSON is invalid

Agent 2: Resource Selector Agent

Purpose: choose learning activities from a small local file.

Uses MCP filesystem to read: `data/study_activities.json`

Example:

```
[
  {"topic": "Java collections", "activity": "Review List, Set, Map", "duration": 20},
  {"topic": "Java collections", "activity": "Solve 5 practice tasks", "duration": 30},
  {"topic": "Java collections", "activity": "Compare ArrayList and LinkedList", "duration": 20},
  {"topic": "Java collections", "activity": "Take a short quiz", "duration": 30}
]
```

Uses:

MCP: `filesystem`

Community tool: `file reader / JSON reader`

Hook: `log tool calls`

Agent 3: Plan Writer Agent

Purpose: create the final study plan.

Output:

Study plan for tomorrow: Java collections

1. 20 min – Review List, Set, Map
 2. 30 min – Solve 5 practice tasks
 3. 20 min – Compare ArrayList and LinkedList
 4. 30 min – Take a short quiz
 5. 20 min – Recap key mistakes
- Total: 2 hours

Uses:

Skill: `format_study_plan`

Interrupt: ask approval before saving plan
Retry: if total time exceeds available time

Multi-agent pattern

Workflow: Extract → Select → Write

Concepts in this example

Skill:

- extract_learning_goal or format_study_plan

MCP:

- filesystem MCP for reading study_activities.json

Steering:

- use only activities from the file
- do not invent time or resources
- ask clarification if time/topic is missing

Community tool:

- file or JSON utility tool

Interrupt:

- approval before saving final plan

Hook:

- log each tool call or each agent step

Retries:

- retry invalid JSON
- retry if total duration is too long

Evaluations:

run_evals.py example

```
import json
from typing import Any

from strands_evals import Case, Experiment
from strands_evals.evaluators import Evaluator
from strands_evals.types import EvaluationResult

from app.study_planner_agent import run_study_planner

class StudyPlanJsonEvaluator(Evaluator):
    def evaluate(self, case: Case, output: Any) -> EvaluationResult:
        failures = []

        try:
            data = json.loads(str(output))
        except json.JSONDecodeError:
            return EvaluationResult{
```

```

score=0.0,
passed=False,
reason="Output is not valid JSON",
)

expected = case.metadata["expected"]
output_text = json.dumps(data, ensure_ascii=False).lower()

if expected["topic"] is not None:
    actual_topic = str(data.get("topic", "")).lower().strip()
    expected_topic = expected["topic"].lower().strip()

    if actual_topic != expected_topic:
        failures.append(
            f"Expected topic '{expected['topic']}', got '{data.get('topic')}'"
        )

    total_minutes = data.get("total_minutes")

    if not isinstance(total_minutes, int):
        failures.append("total_minutes must be an integer")
    elif total_minutes > expected["max_minutes"]:
        failures.append(
            f"Plan is too long: {total_minutes} > {expected['max_minutes']}"
        )

    for phrase in expected["must_include"]:
        if phrase.lower() not in output_text:
            failures.append(f"Missing phrase: {phrase}")

    for phrase in expected["must_not_include"]:
        if phrase.lower() in output_text:
            failures.append(f"Forbidden phrase found: {phrase}")

    if expected.get("requires_clarifying_question"):
        questions = data.get("clarifying_questions", [])

    if not isinstance(questions, list) or len(questions) == 0:
        failures.append("Expected at least one clarifying question")

    passed = len(failures) == 0

    return EvaluationResult(
        score=1.0 if passed else 0.0,
        passed=passed,
        reason="Passed" if passed else "; ".join(failures),
    )

def task_function(case: Case) -> str:
    return run_study_planner(case.input)

def main():
    cases = [
        Case(
            name="java_collections_2_hours",
            input="I have 2 hours tomorrow. I need to study Java collections and prepare for a quiz.",
            metadata={
                "expected": {
                    "topic": "Java collections",
                    "max_minutes": 120,
                    "must_include": ["list", "set", "map", "quiz"],

```

```

"must_not_include": ["spring boot", "docker", "kubernetes"],
},
),
Case(
name="sql_joins_30_minutes",
input="I have only 30 minutes. I need to understand SQL joins.",
metadata={
"expected": {
"topic": "SQL joins",
"max_minutes": 30,
"must_include": ["join"],
"must_not_include": ["java", "docker", "kubernetes"],
}
},
),
Case(
name="unclear_topic",
input="I have 1 hour tomorrow. I need to prepare for class.",
metadata={
"expected": {
"topic": None,
"max_minutes": 60,
"must_include": [],
"must_not_include": [],
"requires_clarifying_question": True,
}
},
),
]

experiment = Experiment(
cases=cases,
evaluators=[StudyPlanJsonEvaluator()],
)

reports = experiment.run_evaluations(task_function)
reports[0].run_display()

if __name__ == "__main__":
    main()

```

Observability:

- logs of agent steps
- tool call count
- retry count
- execution time

Potential project structure

```

study-planner-agent/
|
├── README.md
├── requirements.txt
|
├── app/
|   ├── main.py
|   ├── workflow.py
|   |
|   └── agents/

```

```
| | |— goal_extractor_agent.py
| | |— resource_selector_agent.py
| | |— plan_writer_agent.py
| |
| |— skills/
| | |— extract_learning_goal.py
| | |— format_study_plan.py
| |
| |— hooks/
| | |— logging_hook.py
| |
| |— observability/
| | |— logger.py
| | |— metrics.py
| |
| |— data/
| | |— study_activities.json
| |
| |— evals/
| | |— cases.json
| | |— run_evals.py
| |
| |— outputs/
| | |— study_plan.txt
| |
| |— screenshots/
| | |— terminal_run.png
| | |— logs.png
| | |— aws_cloudwatch.png
```